監査ログシステムに適した指定確認者墨塗署名の実装評価

廣瀬 温貴[1,a)]　駒野 雄一[1,b)]

概要：企業経営が正しく行われていることを社会やステークホルダーに対して示すために、監査は必要不可欠である。近年、ブロックチェーンを利用した監査ログシステムも検討されているが、第三者が閲覧可能なブロックチェーン上に不必要な情報を開示すると企業の不利益につながるため、情報の開示範囲の適切な制御が必要である。一方、監査内容に疑義が生じた場合には裁判所などの信頼できる第三者が非開示とされた情報を確認できることが望ましい。そのような問題に対し、我々は SCIS2025 で一部の情報を秘匿しつつ監査文書の正当性を保証し、問題が発生した場合には信頼できる第三者がその情報を確認できる指定確認者墨塗署名 (*DCSS*) 方式を提案した。本稿は *DCSS* を Java で実装して性能を評価し、その実現可能性を検証する。

キーワード：CSS 2026，指定確認者署名、墨塗署名、指定確認者墨塗署名、監査ログ

Experiments on Designated Confirmer Sanitizable Signature suitable for Audit Log

Haruki Hirose[1,a)]　Yuichi Komano[1,b)]

Abstract: An audit, with which stakeholders of a company are convinced that the company is doing the right thing, is indispensable to modern economic society. Recently, a blockchain-based audit log system has been proposed. However, because information on the blockchain is visible to third parties including rival companies, it is necessary to properly control the scope and content of disclosure. Against such issues, at SCIS 2025, we proposed a designated confirmer sanitizable signature (*DCSS*) scheme that guarantees the legitimacy of an audit document while keeping partial information confidential and allowing a trusted third party to check the validity of the confidential information if some trouble happens. In this paper, we develop a Java implementation of *DCSS* and evaluate its performance to show the feasibility of *DCSS*.

Keywords: CSS 2026, designated confirmer signatures, sanitizable signatures, designated confirmer sanitizable signature, audit log

1. はじめに

一般に、企業の経営はその経営者と従業員だけでなく、取引先や株主を含むステークホルダーにより支えられている。例えば、ステークホルダーは企業が公開している経営状況に基づき、取引や出資を行うか否かを判断する。しかし、公開された経営状況の情報が企業の実態と異なると、ステークホルダーは適正な判断を下すことができずに損害を被る可能性が高まる。そのため、企業活動に対しては、法令や規定の遵守状況や財務状況に関して、評価・報告を行うための監査が行われている。監査には、任意で行う任意監査の他に、法律によって義務付けられている法定監査もあり、企業が経営を続けていく上で監査は避けることができない。

このように、監査は健全な企業活動に必要不可欠であるが、現状では十分にはデジタル化されていない。一般的な監査では、監査人が紙の書類を人の目で一枚ずつ確認しており、一つの企業の監査にも膨大な時間を要している。さ

[1] 千葉工業大学
Chiba Institute of Technology
[a)] s2581035si@chibatech.ac.jp
[b)] yuichi.komano@chibatech.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

らに、監査書類の準備や管理の面でも、紙の書類は過不足なく揃えたり特定の書類を参照したりするための負担が大きい。

そのような中、監査のデジタル化やブロックチェーンの活用も検討されている [6,11]。しかし、監査書類には不必要に開示すると企業の不利益につながる情報も含まれうる。紙の監査書類を用いる場合には、一般向けには秘匿したい箇所を黒く塗りつぶした墨塗文書を開示する方法を用いることができる。しかし、第三者が閲覧可能なブロックチェーン上に墨塗した監査文書を載せると、墨塗した箇所のデータの信頼性が保証されない。この問題に対して、Srivastava ら [10] は、ブロックチェーンと墨塗署名 [1] を組み合わせて、文書の一部を秘匿しつつ堅牢に文書を保管するログシステムの概念を提唱した。しかし、問題が発生した際に墨塗箇所のデータの正しさを確認する手段が存在しなかった。

これらの問題に対し、我々は SCIS2025 において、秘匿すべき機密データを含む監査文書を電子的に保存するログシステムの構築に適した、指定確認者墨塗署名（*DCSS*）方式を提案した [7,12]。これらの文献では、まず、提案方式の登場人物として、署名者（確認者も兼ねる、監査システムにおける企業）、墨塗者（監査システムにおける監査人）、検証者（同じく株主など）、判定者（同じく裁判所）を整理した。そして、それらの登場人物が行う機能として、鍵生成、署名生成、墨塗、署名検証、開示、の 5 つのアルゴリズムをモデル化した。

さらに、一般的構成の定式化に加え、Ateniese ら [2] に基づくカメレオンハッシュと、RSA および AES を組み合わせた指定確認者署名（DCS）を用いた具体的な構成を示した。しかし、*DCSS* の実装評価は未実施であり、その実現可能性の検証が課題であった。

本研究では、理論的な概念提案に留まっていた *DCSS* [7,12] に対し、プログラム実装、実証評価を実施した。本稿の主な貢献は以下のとおりである。

- 提案する *DCSS* を Java を用いて実装し、処理性能評価を実施した。100 回の試行平均に基づく計測の結果、各エンティティの鍵生成（Setup）が高速に行えること、有事の開示判定（Judge）が 1 ブロックあたり約 0.58 ms と軽量であること、および 16 ブロックすべてに墨塗を施すケース（100%サニライズ）でも処理時間が約 102.7 ms に収まることを確認した。すなわち、*DCSS* は実用的な時間で実行可能であると考えられる。

2. 準備

本稿で評価対象とする指定確認者墨塗署名方式の一般的構成は、（通常の）デジタル署名、指定確認者署名、カメレオンハッシュ関数、カメレオンハッシュ関数を利用した墨塗署名を構成要素として利用する。本節では構成要素に関してモデルと安全性を説明する。

2.1 デジタル署名

本節では文献 [5] に従い、デジタル署名のモデルと安全性要件を説明する。

定義 1 (デジタル署名) デジタル署名 *SIG* は以下の 3 個のアルゴリズム $(KGen, Sign, Verify)$ からなる。

- **鍵生成** $KGen$**:**
  セキュリティパラメータ κ を入力とし、署名者の秘密鍵と公開鍵の組 (sk_S, pk_S) を出力する。
- **署名生成** $Sign$**:**
  署名者の秘密鍵 sk_S と文書 $m \in \mathcal{M}$ を入力とし、署名 σ を計算して、(m, σ) の組を出力する。
- **署名検証** $Verify$**:**
  署名者の公開鍵 pk_S と、文書と署名の組 (m, σ) を入力とし、署名が有効であるかの判定するビット $b \in \{0, 1\}$ を出力する。

デジタル署名は、適応的選択文書攻撃（Adaptive chosen message attack）に対して存在的偽造が困難（Unforgeability）であるときに、安全な方式であるという。

2.2 指定確認者署名

本節では文献 [9] に従い、指定確認者署名のモデルと安全性要件を説明する。

定義 2 (指定確認者署名) 指定確認者署名 *DCS* は以下の 4 個のアルゴリズム $(KGen^{DCS}, Sign^{DCS}, Confirm^{DCS}, Deny^{DCS})$ からなる。

- **鍵生成** $KGen^{DCS}$**:**
  セキュリティパラメータ κ を入力とし、署名者の秘密鍵と公開鍵の組 (sk_S, pk_S)、および、確認者の鍵組 (sk_C, pk_C)、検証者の鍵組 (sk_V, pk_V) を出力する。
- **署名生成** $Sign^{DCS}$**:**
  署名者の秘密鍵 sk_S と確認者の公開鍵 pk_C、および、文書 $m \in \mathcal{M}$ を入力とし、署名 σ を計算して、(m, σ) の組を出力する。
- **確認** $Confirm^{DCS}$**:**
  確認者と検証者の対話により実行されるプロトコルである。確認者が用いるアルゴリズム $Confirm_C$ は、$Sign^{DCS}$ で生成された (m, σ) の組、および、署名者と確認者と検証者の公開鍵 (pk_S, pk_C, pk_V)、そして確認者の秘密鍵 sk_C を入力として動作する。検証者が用いるアルゴリズム $Confirm_V$ は、$Confirm_C$ と同様の $(m, \sigma), (pk_S, pk_C, pk_V)$ と、検証者の秘密鍵 sk_V を入力として動作する。これらのアルゴリズムの対話により、署名が有効であるかの判定するビット $b \in \{0, 1\}$ が出力される。ここで、$Confirm_C$ に入力される pk_V と $Confirm_V$ に入力される sk_V は補助入力であり、省略してもよい。

© 2026 Information Processing Society of Japan

- **否認** $Deny^{DCS}$**:**
  確認者と検証者の対話により実行されるプロトコルである。確認者が用いるアルゴリズム $Deny_C$ は、偽造が疑われる文書と署名の組 (m, σ')、および、署名者と確認者と検証者の公開鍵 (pk_S, pk_C, pk_V)、そして確認者の秘密鍵 sk_C を入力として動作する。検証者が用いるアルゴリズム $Deny_V$ は、$Deny_C$ と同様の $(m, \sigma'), (pk_S, pk_C, pk_V)$ と、検証者の秘密鍵 sk_V を入力として動作する。これらのアルゴリズムの対話により、署名が無効であるかの判定するビット $b \in \{0, 1\}$ が出力される。ここで、$Confirm_C$ に入力される pk_V と $Confirm_V$ に入力される sk_V は補助入力であり、省略してもよい。

指定確認者署名は、存在的偽造困難性（Unforgeability）と不可視性（Invisibility）に加えて、確認と否認における対話プロトコルの安全性要件である完全性（Completeness）、健全性（Soundness）、非転用性 （Non-transferability）を満たすときに安全な方式であるという [9]。

2.3 カメレオンハッシュ関数

カメレオンハッシュ関数 [8] とは、秘密鍵と公開鍵のペアを持つ関数であり、秘密鍵なしではハッシュ値が衝突するような別の入力を見つけることは困難であるが、秘密鍵があれば衝突を与える入力を発見することができるという特性がある。この特性を用いることにより、後述する墨塗署名を実現することができる。

当初提案されていたカメレオンハッシュ関数は、衝突を与える二つの入力が明らかになると秘密鍵そのものも計算可能となる方式（鍵漏洩する方式）であった。したがって、ひとたび墨塗前の文書を開示すると、墨塗前後の文書を入手した第三者も秘密鍵を復元して利用することで、ハッシュ値が同じ値となるあたかも墨塗前の文書であるかのような新たな文書を計算できるようになる。そうすると、本来の墨塗前の文書がいずれであるかを識別することができなくなるため、従来の墨塗署名方式では墨塗前の文書を開示できるのは一度のみであった。

Ateniese ら [2] は、そのような鍵漏洩いが生じないカメレオンハッシュ関数の概念と方式を提案した。本節では文献 [2] に従い、カメレオンハッシュ関数のモデルと安全性要件を説明する。

定義 3 (カメレオンハッシュ関数) カメレオンハッシュ関数 CH は以下の 4 個のアルゴリズム $(KGen^{CH}, Hash^{CH}, UForge^{CH}, IForge^{CH})$ からなる。

- **鍵生成** $KGen^{CH}$**:**
  セキュリティパラメータ κ を入力とし、秘密鍵と公開鍵の組 (sk, pk) を出力する。
- **ハッシュ** $Hash^{CH}$**:**
  公開鍵 pk、ラベル L、文書 m とランダムパラメータ r をを入力として、ハッシュ値 C を出力する。
- **衝突生成** $UForge^{CH}$**:**
  公開鍵に紐づけられた秘密鍵 sk とラベル L、文書 m、ランダムパラメータ r を入力とし、$Hash^{CH}(pk, L, m, r) = C = Hash^{CH}(pk, L, m', r')$ となるような文書 m'、およびランダムパラメータ r' を出力する。
- **追加の衝突生成** $IForge^{CH}$**:**
  公開鍵 pk とラベル L と、文書とランダムパラメータの組 $(m, r), (m', r')$ を入力として、新たな衝突を与える文書とランダムパラメータの組 (m'', r'') を出力する。

カメレオンハッシュ関数は、耐衝突性 (Collision-resistance)、意味推察安全性 (Semantic Security)、文書秘匿性 (Message hiding)、鍵漏洩耐性 (Key Exposure Freeness) を満たすときに安全な方式であるという。

2.4 墨塗署名

本節では文献 [3] に従い、墨塗署名のモデルと安全性要件を説明する。

定義 4 (墨塗署名) 墨塗署名 $SANIT$ は、以下の 6 個のアルゴリズム $(KGen^{SANIT}, Sign^{SANIT}, Sanit^{SANIT}, Verify^{SANIT}, Proof^{SANIT}, Judge^{SANIT})$ からなる。

- **鍵生成** $KGen^{SANIT}$**:**
  セキュリティパラメータ κ を入力とし、署名者の秘密鍵と公開鍵の組 (sk_{si}, pk_{si})、および、墨塗者の秘密鍵と公開鍵の組 (sk_{sa}, pk_{sa}) を出力する。
- **署名生成** $Sign^{SANIT}$**:**
  署名者の秘密鍵 sk_{si} と墨塗者の公開鍵 pk_{sa}、墨塗可能箇所の情報 ADM、および文書 $m \in \mathcal{M}$ を入力とし、署名 σ を出力する。
- **墨塗** $Sanit^{SANIT}$**:**
  文書 m と署名 σ、署名者の公開鍵 pk_{si}、墨塗者の秘密鍵 sk_{sa}、および墨塗可能箇所の情報 ADM を入力とし、新たに墨塗済みの文書 $\widetilde{m}$ とそれに対応する署名 $\widetilde{\sigma}$ を出力する。
- **検証** $Verify^{SANIT}$**:**
  文書 m と署名 σ、署名者の公開鍵 pk_{si}、墨塗者の公開鍵 pk_{sa} を入力とし、署名が有効であるか否かの判定結果 $\{0, 1\}$ を出力する。なお、墨塗後に $Verify^{SANIT}(\widetilde{m}, \widetilde{\sigma}, pk_{si}, pk_{sa})$ が入力された場合においても同様に署名が有効であるか否かの判定結果 $b \in \{0, 1\}$ を出力する。
- **証拠生成** $Proof^{SANIT}$**:**
  署名者の秘密鍵 sk_{si}、文書 m と署名 σ、墨塗者の公開鍵 pk_{sa} を入力とし、証拠 π を出力する。
- **判定** $Judge^{SANIT}$**:**
  文書 m と署名 σ、署名者の公開鍵 pk_{si}、墨塗者の公開鍵 pk_{sa}、証拠 π を入力とし、署名が誰によって作成されたかを示す判定結果 $d \in \{Sig, San\}$ を出力する。このアルゴリズムも $Verify^{SANIT}$ と同様に、$Judge^{SANIT}(\widetilde{m}, \widetilde{\sigma}, pk_{si}, pk_{sa}, \pi)$ が入力された場合にも $d \in \{Sig, San\}$ を出力する。

墨塗署名は、存在的偽造が困難 （Unforgeability）、不

© 2026 Information Processing Society of Japan

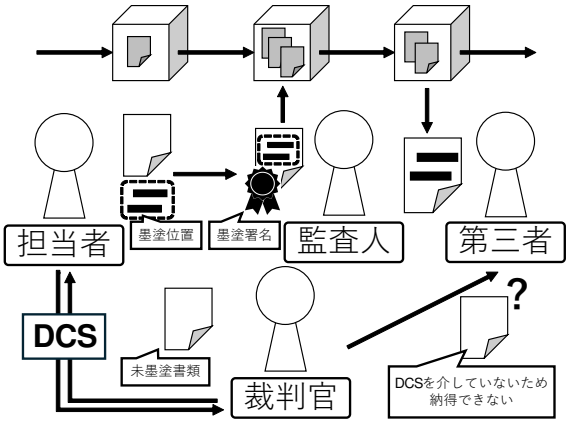


図 1 想定システム

変性 (Immutability)、プライバシー (Privacy)、透明性 (Transparency)、責任追跡性 (Accountability) を満たすときに安全な方式であるという。

3. 指定確認者墨塗署名

本節では、文献 [7, 12] で提案した指定確認者墨塗署名 *DCSS* のモデルと安全性要件を紹介する。

3.1 登場人物

DCSS は以下のエンティティで実行される。

- **企業 Company:**
  監査書類の作成、およびその書類の墨塗可能箇所を指定したのち、署名を発行する。
- **監査人 Auditor:**
  企業からの監査書類および署名の組を受け取り、指定された箇所の墨塗を実施する。そして、墨塗後の書類および署名の組をブロックチェーン上に記録するなど、第三者が閲覧できるようにする。
- **第三者 Viewer:**
  ブロックチェーン上に記録されている墨塗済みの書類と署名の組を取得する。そして、墨塗済みの書類と署名の組を検証することにより、書類の正しさを確認する。
- **裁判所 Judge:**
  監査内容に疑義が生じた場合には、裁判所など信頼のおける機関が非開示とされている墨塗箇所の本来の情報を確認することができる。

3.2 モデル

次に、*DCSS* のモデルを紹介する。まず、図 1 を用いて、*DCSS* の処理の概要を説明する。

まず、企業の担当者は監査対象の文書を自らの署名とともに監査人に提出し、第三者に対して内容を秘匿したい墨塗位置を指定する。それらを受け取った監査人は、文書の内容を監査し、内容に問題がなければ指定された位置を秘匿した墨塗署名を作成して、ブロックチェーンに保存する。株主などの第三者はブロックチェーンから墨塗署名を入手して、監査が正しく行われていることを確認することができる。

後に監査内容に疑義が生じた場合には、裁判官などの信頼できる公的な機関が企業担当者や監査人から墨塗される前の元の情報を入手して、元の文書の内容を確認する。ただし、元の文書の確認には指定者確認署名を用いることとする。このとき、元の文書の正しさを納得できるのは裁判官のみであり、その文書の正しさは第三者には確認できないようにする。そのようにすることで、墨塗前の文書の一人歩きを防止する。

このような処理に対し、*DCSS* のモデルは以下のように与えられる [7, 12]。

定義 5 (指定確認者墨塗署名) 署名者 (確認者の役割も果たす、ここでは企業の担当者など)、墨塗者 (ここでは監査人)、検証者 (ここでは株主など)、判定者 (ここでは裁判官など) は、以下の 5 個のアルゴリズム ($KGen^{DCSS}$, $Sign^{DCSS}$, $Sanit^{DCSS}$, $Verify^{DCSS}$, $Disclosure^{DCSS}$) を実行する。ここで、墨塗を行わないメッセージブロックのインデックスの集合を $I = \{i_1, i_2, \cdots\}$ であらわし、墨塗が可能であるメッセージブロックのインデックスの集合を $\widetilde{I} = \{\widetilde{i_1}, \widetilde{i_2}, \cdots\}$ であらわすものとする。

- **鍵生成** $KGen^{DCSS}$**:**
  セキュリティパラメータ κ を入力として、署名者 (確認者も兼ねており、ここでは企業の社員など) の秘密鍵と公開鍵の組 (sk_S, pk_S)、および、確認者 (ここでは企業の社員) の鍵組 (sk_C, pk_C)、墨塗者の鍵組 (sk_{si}, pk_{si})、判定者 (ここでは裁判所) の鍵組 (sk_V, pk_V) を出力する。
- **署名生成** $Sign^{DCSS}$**:**
  文書 m_i、署名者の秘密鍵 sk_S、確認者の公開鍵 pk_C、墨塗者の公開鍵 pk_{si} を入力として、文書と署名の組 (m, σ) と墨塗可能箇所の情報 ADM を出力する。
  また、後に墨塗を行った行為者の特定を可能とするための証拠 π も計算して出力する。
- **墨塗** $Sanit^{DCSS}$**:**
  文書 m と署名 σ、署名者の公開鍵 pk_S、確認者の公開鍵 pk_C、墨塗者の秘密鍵 sk_{si}、および墨塗可能箇所の情報 ADM を入力として、一部を墨塗した文書 $\widetilde{m}$ と署名 $\widetilde{\sigma}$ の組を出力する。
- **署名検証** $Verify^{DCSS}$**:**
  文書 $\widetilde{m}$ と署名 $\widetilde{\sigma}$、署名者の公開鍵 pk_S、確認者の公開鍵 pk_C、墨塗者の公開鍵 pk_{si} を入力として、署名が有効であるか否かの判定結果 $Verify^{DCSS}(\widetilde{m}, \widetilde{\sigma}, pk_S, pk_C, pk_{si}) = b \in \{0, 1\}$ を出力する。
  また、文書 m、署名 σ、署名者の公開鍵 pk_S、確認者の公開鍵 pk_C、墨塗者の公開鍵 pk_{si}、$Sign^{DCSS}$ にて計算した証拠 π を入力として、署名が誰によって作成されたのかの

© 2026 Information Processing Society of Japan

判定結果 $d \in \{Sig, San\}$ を出力する。これにより、検証者は墨塗を行った行為者が監査人であるかを確認できる。

- **開示 $Disclosure^{DCSS}$:**
  確認者と判定者の対話により実行される。確認者は $Confirm_C$、判定者は $Confirm_V$ を用いる。確認者が用いるアルゴリズム $Confirm_C$ は、$Sign^{DCS}$ で生成された (m, σ) の組（m は墨塗前の文書）、および、署名者と確認者、墨塗者、検証者の公開鍵の組 $(pk_S, pk_C, pk_{si}, pk_V)$、そして確認者の秘密鍵 sk_C を入力として動作する。判定者が用いるアルゴリズム $Confirm_V$ は、$Sign^{DCS}$ で生成された (m, σ) の組（m は墨塗前の文書）、および、署名者と確認者、墨塗者、検証者の公開鍵の組 $(pk_S, pk_C, pk_{si}, pk_V)$、そして検証者の秘密鍵 sk_V を入力として動作する。これらのアルゴリズムの対話により、σ 署名が有効であるかを判定するビット $b \in \{0, 1\}$ が出力される。

なお、上述のモデルでは、署名者（企業の社員など）に確認者の役割を持たせて説明をしているが、墨塗者（監査人）に確認者の役割を持たせて方式を構成することも可能である。

3.3 安全性要件

指定確認者墨塗署名 *DCSS* の安全性要件は以下のように定義される。

- **不変性 Immutability:**
  墨塗者が、許可されていない変更を文書に対して加えた場合に、その文書に対して有効な署名を生成することができない。
- **プライバシー Privacy:**
  墨塗後のデータや署名などから、墨塗前の文書に関する情報は漏れない。
- **責任追跡性 Accountability:**
  誰がどの文書に対して署名を発行したのかを正確に証明できる。この性質により、署名者が発行した文書に対して墨塗者は責任を追う必要がなく、墨塗者が発行した文書に対しても署名者は責任を追う必要はない。
- **存在的偽造困難性 Existential Unforgeability:**
  署名者の秘密鍵を知らずに、検証に合格する署名を生成することはできない。
- **署名の不可視性 Invisiblity of Signatures:**
  確認者と判定者の間での会話によってのみ署名の有効性が確認でき、第三者は署名が有効であるか否かに関する情報を得ることはできない。

4. 実装評価

本節では、指定確認者墨塗署名 *DCSS* を Java 言語により実装し、その性能を測定した結果を示す。

表 1 実験に用いた評価マシンの仕様

項目	実装仕様
CPU	Apple M4 (10 Cores)
Memory	32.0 GB RAM
Runtime	Amazon Corretto 25.0.3 LTS (build 25.0.3+9-LTS)
Language	Java 25

表 2 各エンティティにおける Setup 平均処理時間（100 回平均）

エンティティ	平均処理時間 (ms)
System	0.5461
Company (Signer)	49.7819
Auditor (Sanitizer)	0.0145
Judge (Confirmer)	52.6618

4.1 実装仕様と実験環境

提案方式の各アルゴリズムの性能を測定するため、Java を用いてプログラムを作成した。実験における計測実行環境を表 1 に示す。

DCSS の具体的構成では、文献 [1, 4] の方式に基づき、RSA に基づくカメレオンハッシュ関数と墨塗署名、離散対数問題に基づく指定確認者署名、ハッシュ関数を利用する。本稿の評価実験では、RSA と離散対数問題は 2048 ビットの公開情報、ハッシュ関数は SHA-256 を使用した。

4.2 初期鍵生成（Setup）処理時間

プロトコルの最初期に、一括して一度だけ実行される各ロール（エンティティ）ごとの初期セットアップおよび鍵生成処理の平均実行時間を測定した。計測は、それぞれ 100 回の試行から平均処理時間（ミリ秒：ms）を算出した。結果を表 2 に示す。

表 2 に示したとおり、Company（Signer）および Judge（Confirmer）のセットアップ時間はそれぞれ約 49.8 ms、52.7 ms と、実運用上十分に軽量であった。また、Auditor（Sanitizer）のカメレオンハッシュ用の鍵ペア生成は、0.0145 ms と高速な処理された。

4.3 各アルゴリズムの処理時間

DCSS の署名生成処理や墨塗などの処理時間は、文書の総ブロック数や墨塗を許可するブロック数に依存すると考えられる。そこで、本稿では監査ログ文書全体の総ブロック数 n （$n = 1, 2, 4, 8, 16$）と、そのうち企業が初期署名時に監査人へ墨塗（サニライズ）を許可するブロック数 m を変化させて、以下の 4 つの基本操作の処理時間を計測した。その結果を表 3 に示す。

(1) 署名生成 ($Sign^{DCSS}$)
(2) 墨塗 ($Sanit^{DCSS}$)
(3) 検証 ($Verify^{DCSS}$)
(4) 開示 ($Disclosure^{DCSS}$)

© 2026 Information Processing Society of Japan

表 3 ブロック数 n および墨塗数 m を変化させた場合の平均処理時間 (ms) の網羅測定結果

ブロック数 n	墨塗数 m	署名生成 (ms)	墨塗 (ms)	検証 (ms)	開示 (ms)	合計 (ms)
1	0 (0%)	0.5905	N/A	0.0315	N/A	0.6553
1	1 (100%)	1.6769	2.2473	2.0192	0.5833	7.0479
2	0 (0%)	0.5649	N/A	0.0233	N/A	0.6120
2	1 (50%)	1.6512	2.2340	2.0105	0.5692	6.9729
2	2 (100%)	2.7256	4.4611	3.9863	1.1315	13.2928
4	0 (0%)	0.5672	N/A	0.0228	N/A	0.6133
4	2 (50%)	2.7269	4.4622	4.0032	1.1382	13.3183
4	4 (100%)	4.8829	8.9443	7.9766	2.2712	26.0359
8	0 (0%)	0.5711	N/A	0.0231	N/A	0.6174
8	4 (50%)	4.9136	8.9722	8.0114	2.3461	26.2116
8	8 (100%)	9.2532	17.9554	16.0112	4.5573	51.6864
16	0 (0%)	0.5747	N/A	0.0232	N/A	0.6216
16	8 (50%)	9.2514	17.9565	15.9970	4.5584	51.6748
16	16 (100%)	17.9374	35.9146	31.9649	9.1241	102.7417

5. 考察

本節では、4 節の評価結果について考察する。

まず、初期鍵生成にかかる時間について考察する。初期鍵生成で生成される鍵組は、従来の RSA や離散対数問題系の公開鍵暗号に用いる鍵組と同じであり、実用的な時間で生成されることが期待される。実際、表 2 の結果から、各エンティティが必要とする鍵組の生成に要する時間は高々 50ms 程度であり、十分に実用的な時間で生成できている。

次に、署名生成、墨塗、検証、開示の各アルゴリズムにかかる時間について考察する。*DCSS* は、ブロックごとに署名の生成、墨塗、検証、開示を行う。そのため、総ブロック数 n および墨塗数 m に応じて、処理時間が変動すると考えられる。実際、表 3 において、署名生成は同一の m に対して n に比例して処理時間が増加することが確認できた。また、同一の n に対しても m に比例して署名生成の処理時間が増加するが、これは後の墨塗を可能とするために m 個のブロックにカメレオンハッシュ関数の処理を適用したためと考えられる。墨塗、検証、開示に関しても、同様の傾向が確認された。また、本実験における最悪のケースである、総ブロック数 $n = 16$ のすべてに対して監査人が墨塗を施した（100%サニライズ時）の場合においても、合計の処理時間は 102.74 ms 程度であり、現実的かつ実用的な処理時間で達成できることを確認した。

このことから、提案方式を監査ログシステムへ応用しても、パフォーマンス上の致命的なボトルネックとはならないと考えられる。

6. まとめ

本稿では、監査ログシステムへの応用を想定して、一部の情報を秘匿しながらも監査文書の正当性を保証し、問題が発生した場合には信頼できる第三者がその情報を確認できる指定確認者墨塗署名方式 [7, 12] を Java 実装し、処理性能を評価して実用性を検証した。

その結果、各プロトコルの実行時間は墨塗りブロック数 m に対する線形性（$O(m)$）を示すことを確認した。また、16 ブロックすべてを墨塗する状況においても、全体のトータル処理時間は約 102.7 ms と高速に動作し、懐疑が生じた場合に裁判所が墨塗文書の内容を正当に検証する処理においても 1 ブロックあたり約 0.58 ms と高速であることを確認した。

処理の最適化による性能向上と、ポスト量子計算機時代における長期的安全性を保証するため、ポスト量子墨塗り署名 [10] などを *DCS* と統合して実装評価することが今後の課題である。

謝辞: 本研究は JSPS 科研費 JP24K14951 の助成を受けたものです。

参考文献

[1] Giuseppe Ateniese, Daniel H. Chou, Breno de Medeiros, and Gene Tsudik. Sanitizable signatures. In Sabrina De Capitani di Vimercati, Paul F. Syverson, and Dieter Gollmann, editors, *Computer Security - ESORICS 2005, 10th European Symposium on Research in Computer Security, Milan, Italy, September 12-14, 2005, Proceedings*, volume 3679 of *Lecture Notes in Computer Science*, pages 159–177. Springer, 2005.

[2] Giuseppe Ateniese and Breno de Medeiros. On the key exposure problem in chameleon hashes. In Carlo Blundo and Stelvio Cimato, editors, *Security in Communication Networks, 4th International Conference, SCN 2004, Amalfi, Italy, September 8-10, 2004, Revised Selected Papers*, volume 3352 of *Lecture Notes in Computer Science*, pages 165–179. Springer, 2004.

[3] Christina Brzuska, Marc Fischlin, Tobias Freudenreich, Anja Lehmann, Marcus Page, Jakob Schelbert, Dominique Schröder, and Florian Volk. Security of sanitizable signatures revisited. In Stanislaw Jarecki and Gene Tsudik, editors, *Public Key Cryptography - PKC 2009,*

© 2026 Information Processing Society of Japan

12th International Conference on Practice and Theory in Public Key Cryptography, Irvine, CA, USA, March 18-20, 2009. Proceedings, volume 5443 of *Lecture Notes in Computer Science*, pages 317–336. Springer, 2009.

[4] David Chaum. Designated confirmer signatures. In Alfredo De Santis, editor, *Advances in Cryptology - EUROCRYPT '94, Workshop on the Theory and Application of Cryptographic Techniques, Perugia, Italy, May 9-12, 1994, Proceedings*, volume 950 of *Lecture Notes in Computer Science*, pages 86–91. Springer, 1994.

[5] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM J. Comput.*, 17(2):281–308, 1988.

[6] Hongdan Han, Radha K. Shiwakoti, Robin Jarvis, Chima Mordi, and David Botchie. Accounting and auditing with blockchain technology and artificial intelligence: A literature review. *Int. J. Account. Inf. Syst.*, 48:100598, 2023.

[7] Haruki Hirose and Yuichi Komano. Designated confirmer sanitizable signature suitable for audit log. In *Thirteenth International Symposium on Computing and Networking, CANDAR 2025 - Workshops, Yamagata, Japan, November 25-28, 2025*, pages 238–244. IEEE, 2025.

[8] Hugo Krawczyk and Tal Rabin. Chameleon hashing and signatures. *IACR Cryptol. ePrint Arch.*, page 10, 1998.

[9] Jean Monnerat and Serge Vaudenay. Chaum's designated confirmer signature revisited. In Jianying Zhou, Javier López, Robert H. Deng, and Feng Bao, editors, *Information Security, 8th International Conference, ISC 2005, Singapore, September 20-23, 2005, Proceedings*, volume 3650 of *Lecture Notes in Computer Science*, pages 164–178. Springer, 2005.

[10] Vikas Srivastava, Paresh Baidya, Sumit Kumar Debnath, and Sihem Mesnager. Blockchain-envisioned post-quantum secure sanitizable signature for audit logs management. *CoRR*, abs/2312.16322, 2023.

[11] 梅村圭輝. ブロックチェーンを利用した不正会計の防止. **早稲田大学大学院会計研究科 テーマ研究論文** *(2016* **年度***)*, 2017.

[12] 廣瀬 温貴, 駒野 雄一. 監査ログシステムに適した指定確認者墨塗署名. *2025* **年暗号と情報セキュリティシンポジウム** *(SCIS 2025), 3C3-3*, 2025.

© 2026 Information Processing Society of Japan